Agentic Attack Surface Management: Research and Content Plan

Cybersecurity · 3 days (8 hours per day) · Intermediate · Compiled 30 October 2026

This document is the working plan for building the course from its outline. It records where the course came from, why the outline is shaped the way it is, what goes into every module (lessons, teaching points, demo, lab, pitfalls and knowledge checks), the lab environment to prepare, and the sources behind each fact. Numbers in square brackets point to the reference list at the end; every URL there was opened and checked against the fact it supports.

- 1. Where this course came from
- 2. How the outline was shaped
- 3. Audience and prerequisites
- 4. Lab environment
- 5. Module-by-module content plan
- 6. Capstone plan
- 7. Promotional video: research behind the script
- References

1. Where this course came from

Client input.

> From 'Agentic Cybersecurity Course Prototypes' (Course 3, Agentic Attack Surface Management): "A platform that consolidates approved asset sources, identifies changes and inventory gaps, links assets to owners and known findings, and prioritizes exposure review. Active discovery is constrained to explicitly authorized domains and ranges." The build sequence: establish authorization and scope (allowlist of lab-owned domains and synthetic assets, exclusions, rate limits, audit trail; disable arbitrary target entry); fixture-based adapters for DNS, certificate metadata, cloud inventory, CMDB and scanner findings; canonical asset IDs with source, first/last seen, confidence, environment and relationships; deterministic entity resolution with uncertain matches reviewable; snapshot change detection; exposure correlation distinguishing observed facts from inferred exposure; ownership from CMDB, tags and repository metadata with manual correction; transparent ranking and review tickets via a mock connector; metrics for entity-link precision, unowned asset rate, stale-asset classification and change-detection accuracy. Safety note: "Asset discovery can touch systems outside a customer's authority if scope is weak."

Attack surface management starts from a simple problem: organizations do not know everything they expose. Hostnames appear in Certificate Transparency logs as soon as certificates are issued [20][3], cloud resources are created and deleted daily across accounts and subscriptions [1][2], and DNS records outlive the resources they point to, creating subdomain takeover risk [9]. Shorter certificate lifetimes, falling toward 47 days by 2029, add more change to track [21].

Public policy treats asset visibility as a baseline. CISA's BOD 23-01 requires federal agencies to run automated asset discovery every seven days [13], and BOD 26-04 makes public exposure a direct input to remediation deadlines and requires tagging of externally reachable assets [22]. Building that

inventory means reconciling sources that disagree, are eventually consistent and only show what the caller can read [2][10].

The same tools that find assets can easily reach systems an organization does not own. That is why this course puts authorization first, following the rules-of-engagement model of NIST SP 800-115 [6], and keeps scope enforcement, ranking and ownership logic deterministic. The agent's role is narrow: summarize exposure from supplied evidence and keep observed facts separate from inference, which OWASP's guidance on excessive agency and misinformation supports [14][23].

2. How the outline was shaped

Decision	Rationale
Scope policy is the first module and every adapter must pass through one gate.	The client's build sequence starts with authorization and its safety note names weak scope as the main risk; NIST's ROE model gives a structure learners can reuse [6].
All five sources are fixture-based; lab domains use reserved names such as example. and test.	The client requires fixture sources and lab-owned domains; reserved names cannot be registered by third parties [11].
Compressed the four-week outline into three days by following the nine-step build sequence one module per step.	Each step yields a tested increment; the client's week-four change monitoring moved to day two because ownership and ranking depend on it.
Entity resolution is precision-first with a review queue; nothing is silently merged.	Direct from the client: 'Keep uncertain matches reviewable' and 'precision-oriented tests'. A false merge corrupts ownership and exposure downstream.
The agent only summarizes exposure; it has no discovery tools.	Avoids excessive agency [14]; the client asks to 'distinguish observed facts from inferred exposure' and to review for 'unsupported exposure claims'.
Tickets are review tickets through a mock connector; the platform never changes DNS or cloud resources.	The client specifies a mock connector and human review for uncertain ownership or exposure; changes to production need separate approval.

3. Audience and prerequisites

Role	What they need	Starting point
External attack surface analyst	A defensible, scoped inventory and change feed instead of ad hoc recon.	Knows DNS, TLS and recon tools; moderate Python.
Security operations engineer	Prioritized exposure review queue with owners.	Knows alerts and ticketing; less inventory work.
Cloud security engineer	Reconcile cloud inventory with DNS and certificates; detect dangling records.	Knows one cloud provider's inventory tools.
IT asset manager	Close CMDB gaps with evidence and ownership suggestions.	Knows CMDB processes; little scripting.
MSSP engineer	Reusable scope enforcement and audit for multi-client work.	Strong tooling skills; needs strict scope discipline.

Prerequisites from the outline

- DNS, HTTP and TLS fundamentals
- Asset inventory concepts
- Python
- Basic cloud and network knowledge

Checks before enrolment

- Can you explain A, AAAA, CNAME and TXT records and what TTL means?
- Can you read a TLS certificate and find its SANs, issuer and expiry?
- Can you write Python that loads JSON and compares two lists of dicts?
- Do you know what a CIDR range is and whether 10.0.0.0/8 is private?
- Have you used one cloud console to list resources and tags?
- Can you run docker compose up and read logs?

Course outcomes

- Build a normalized asset inventory from approved sources with explicit scope enforcement
- Reconcile domains, certificates, cloud resources, services and ownership records
- Detect inventory changes and potential stale or orphaned assets
- Correlate assets with vulnerability and exposure findings, separating observed facts from inferred exposure
- Generate evidence-backed ownership and remediation tasks while blocking out-of-scope discovery

4. Lab environment

GenNoorTech INNOVATE · BUILD · TRAIN for more info: https://gennoor.com

A Docker Compose stack per learner: FastAPI application, PostgreSQL, fixture servers for DNS, CT, cloud inventory (AWS, Azure, GCP shapes), CMDB and scanner findings, a Jira-shaped ticket mock, and a simple web UI for the review queue and asset graph. No real cloud accounts or internet discovery are used. Model calls go through an adapter with a recorded-response mode for offline runs.

Components

Gennoor Tech
TRAIN · INNOVATE · BUILD
For more info - https://gennoor.com

Component	Version or edition	Purpose	How to obtain	Notes
Python	3.12 or later	Application, adapters, tests	python.org or the provided dev container	ipaddress module used for scope checks [16].
FastAPI and Pydantic v2	Current releases, pinned	API, review queue, schemas	PyPI	Strict models for adapter results.
PostgreSQL	18 (official postgres:18 image)	Inventory, snapshots, audit, review queue	Docker Hub	Append-only audit table.
Docker Engine with Compose v2	Current stable	Repeatable lab stack	Docker Engine on Linux or Docker Desktop (check licensing)	Cloud VM image for locked-down laptops.
Fixture servers	Course-provided containers	DNS zone export, CT get-entries/get-sth shapes, cloud inventory exports, CMDB, scanner	Provided in the starter repository	CT fixture mirrors RFC 6962 endpoints [3]; cloud fixtures mirror AWS Config, Azure Resource Graph and Cloud Asset Inventory identifiers [1][2][10].
Python Public Suffix List library	Current release with a bundled, dated PSL snapshot	Registrable domain checks in scope policy	PyPI; snapshot committed to the repo	Keep the snapshot fixed for deterministic tests [17].
Jira-shaped ticket mock	Course-provided container	Review ticket creation and JQL-style search	Provided; optional free Jira Cloud site	Same REST shape as Jira examples [25].
LLM API access	Provider-agnostic adapter; Claude API used in delivery	Exposure summary agent	Instructor proxy with organization key; recorded responses offline	JSON schema outputs, stop_reason checked [24].
Optional real sources (instructor demo only)	AWS Config, Azure Resource Graph, Cloud Asset Inventory in a course-owned sandbox account	Show real API shapes	Course-owned accounts with read-only roles	Never used for learner discovery.

Fixtures and sample data

Fixture	Contents	Used in
Scope policy and ROE	Three lab domains under example., two lab cloud accounts, IP ranges, 14 excluded ranges, look-alike and partner domains for negative tests.	Authorization, scope policy and the lab
Source fixtures	DNS zone exports with CNAME chains and planted dangling records; CT entries with SANs, issuers and validity; AWS/Azure/GCP inventory exports; CMDB CIs; scanner findings.	Source adapters; Asset normalization and canonical IDs
Labeled link set	Record pairs labeled same, different or ambiguous, including shared CDN IPs and renamed hosts.	Entity resolution
Snapshot pairs with answer keys	10-27/10-28 and capstone pairs with expected new, changed, missing, stale and orphaned classes, plus a source-outage trap.	Change detection; Capstone
Exposure evaluation set	Assets with findings, services and labeled observed/inferred claims, including misleading text.	Exposure correlation with an agent
Ownership fixtures	CMDB owner groups, cloud tags with case variants, CODEOWNERS files, and an answer key with planted wrong suggestions.	Ownership resolution; Prioritization, ticketing and inventory quality metrics

Lab safety rules

- No learner action reaches the internet; all adapters read fixtures, and outbound network is blocked in the lab compose network.
- Arbitrary target entry is removed from the CLI and API in the course build.
- Lab names use reserved domains (example., test.) only; never real company domains in slides or fixtures.
- Instructor demos of real cloud APIs use course-owned accounts with read-only roles.
- Any extension to active checks requires a written ROE with exclude list, hours and halt criteria [6].

5. Module-by-module content plan

Day 1

Module 1: Authorization, scope policy and the lab

Goal. Implement an explicit, code-enforced scope policy with allowlists, exclusions, rate limits and an audit trail, and prove that out-of-scope targets are blocked and logged.

Outline topics.

- Explicit allowlist of lab-owned domains and synthetic assets, exclusions, rate limits, audit trail
- Arbitrary target entry is disabled in the course environment
- Why weak scope is the biggest risk in attack surface tooling

Learning objectives.

- Learners can explain why weak scope is the biggest risk in attack surface tooling and how it maps to rules of engagement.
- Learners can write a scope policy covering domains, IP ranges, cloud accounts, exclusions and rate limits.
- Learners can enforce the policy in a single gate that every adapter must call, outside any model.
- Learners can write tests that prove blocked targets never reach an adapter and are logged.

Why scope comes first

- Discovery can easily touch systems you do not own: shared hosting, CDNs, partners, previous owners of an IP.
- NIST SP 800-115 defines rules of engagement as constraints set before testing that give authority for defined activities only; its template includes authorized and unauthorized addresses and an explicit exclude list [6].
- The same template sets test hours, points of contact and halt criteria; the policy file carries these too [6].
- Even passive information gathering is listed as testing activity in OWASP WSTG 4.1, so it still needs authorization [8].
- The client input requires arbitrary target entry to be disabled in the course environment.

The scope policy file

- Fields: policy_id, version, approver, valid_from/to, domains (with include_subdomains flag), ip_ranges, cloud_accounts, exclusions, rate_limits per adapter, allowed_hours, contacts.
- Use reserved names for lab domains: example. is reserved for documentation and cannot be registered, and test. names are answered negatively by caching resolvers by default [11].
- Domain matching: normalize to lowercase, strip trailing dot, compare label by label; never match by substring ('lab.example' must not match 'evillab.example').
- Check registrable boundaries with the Public Suffix List so a policy can never accidentally allow a whole public suffix such as co.uk [17].
- IP matching: ip_address(x) in ip_network(y), with ip_network strict so ranges with host bits set are rejected as typos [16].

Enforcement outside the model

- One ScopeGate.check(target, action, adapter) call before every network or API action; adapters cannot be constructed without the gate.
- OWASP excessive agency guidance: narrow tools, no open-ended URL tool, least-privilege credentials, and complete mediation in downstream systems [14].

- The agent can propose a target; the gate decides. Rejected proposals are logged as scope events.
- Exclusions win over inclusions; an excluded range inside an allowed range is blocked.
- Reject non-global addresses unless explicitly listed; is_global and is_private reflect IANA registries, and 100.64.0.0/10 is neither [16].

Rate limits and audit trail

- Per-adapter token bucket with limits from the policy; exceeding returns a throttled result, not an exception that retries forever.
- Audit events: audit_id, at, actor, adapter, target, action, decision (allowed, blocked, throttled), policy_version, reason.
- Audit logs are append-only and written before the action runs.
- Rate limiting and logging are also OWASP's damage-limiting measures for agentic systems [14].

Instructor demo.

1. Walk through the NIST ROE template sections and map each to a policy field [6].
2. Show the policy YAML with lab domains under example. and an exclusion list [11].
3. Run asm run with an in-scope target and show the audit entry.
4. Try partner-corp.example and evillab.example and show both blocked and logged.
5. Try a range with host bits set and show policy load fail [16].
6. Show the agent proposing an out-of-scope target and the gate rejecting it.

Hands-on lab. Implement the scope policy and prove that out-of-scope targets are blocked and logged.

1. Read docs/rules-of-engagement.md (course-provided) and fill in contacts and allowed hours.
2. Write scope/policy.yaml with three lab domains, two lab cloud accounts, IP ranges and exclusions.
3. Implement the policy loader with validation (strict networks, lowercase domains, no public suffixes).
4. Implement ScopeGate.check with label-wise domain matching and exclusion precedence.
5. Implement per-adapter rate limits.
6. Implement the append-only audit log in PostgreSQL.
7. Write tests: subdomain allowed, look-alike blocked, excluded range blocked, IPv6 handling, throttling.
8. Remove any free-text target input from the CLI and API; targets come only from policy and approved sources.
9. Run the test suite and commit the audit sample.

Lab success criteria.

- Every out-of-scope test target is blocked and has an audit entry.

GenNoor Tech
For more info https://gennoor.com

- No code path reaches an adapter without passing the gate (checked by a test that patches the gate to deny all).
- Policy load fails on malformed ranges and public suffix entries.
- Arbitrary target entry is not possible in the course build.

Deliverable. Scope policy with audit.

Common pitfalls.

- Substring or endswith matching without a label boundary.
- Letting exclusions be overridden by more specific inclusions.
- Writing the audit entry after the action, so failures leave no trace.
- Giving the agent a generic fetch-URL tool.
- Using real company domains in lab examples.

Knowledge check.

1. What does an ROE give a testing team, according to NIST? Answer: Authority to conduct defined activities without further permission, within stated constraints such as an exclude list [6].
2. Why use example. or test. names for lab domains? Answer: They are reserved and cannot be registered, so lab traffic never reaches a real third party [11].
3. What happens with ip_network('10.0.0.1/24') by default? Answer: It raises ValueError because host bits are set; strict is the default [16].
4. Where must authorization be enforced? Answer: In application code and downstream systems, not by the model [14].

Slides and assets to prepare.

- Rules of engagement template adapted for the lab.
- Scope policy YAML with annotated fields.
- Domain matching test table (allowed, look-alike, excluded).
- Audit event schema.
- Scope gate sequence diagram.

Sources for this module. 6, 8, 11, 14, 16, 17

Module 2: Source adapters

Goal. Build five fixture-based adapters (DNS, certificate metadata, cloud inventory, CMDB, scanner findings) behind one adapter contract with timeouts, retries and least privilege.

Outline topics.

- Fixture-based adapters for DNS records, certificate metadata, cloud inventory, CMDB entries and scanner findings
- Adapter contract, timeouts, retries and least privilege

GenNoor Tech TRAINING · LIVE · BUILD · For more info - https://gennoor.com

Learning objectives.

- Learners can define an adapter contract that returns typed records with provenance and explicit errors.
- Learners can describe what each real source provides and its limits (consistency, history, permissions).
- Learners can implement fixture adapters that mirror the shape of real APIs.
- Learners can apply timeouts, bounded retries and least-privilege credentials.

The adapter contract

- fetch(scope_gate, snapshot_id) returns AdapterResult(records, errors, source, fetched_at, source_version, partial: bool).
- Every call goes through the scope gate first; the adapter receives a gate, not raw targets.
- Timeouts per call, retries with exponential backoff and a maximum count, then a recorded error, never an endless loop.
- Partial results are flagged: Azure Resource Graph silently returns only resources the caller can read, so a 'complete' flag cannot be assumed [2].
- Credentials are read-only and scoped to the lab accounts; tools are narrow per OWASP excessive agency guidance [14].

DNS and certificate sources

- DNS fixtures model zone exports: name, type, value, TTL, zone; RR order carries no meaning, so sort before hashing [5].
- A CNAME should not coexist with other data at the same name; flag violations as data quality issues [5].
- Certificate Transparency logs are append-only public logs; real readers use /ct/v1/get-sth and /ct/v1/get-entries [3].
- A CT monitor must inspect every new entry in every log it watches, which is why production monitors track log positions [4].
- Let's Encrypt submits every certificate it issues to CT logs, so new hostnames often appear in CT before anyone updates the inventory [20].

Cloud inventory sources

- AWS Config records configuration and relationships over time and aggregators combine accounts and Regions [1].
- Azure Resource Graph uses KQL across subscriptions, is eventually consistent and throttles per user (x-ms-user-quota-remaining) [2].
- Google Cloud Asset Inventory exports to BigQuery or Cloud Storage, keeps 35 days of history and is not meant for live queries [10].
- Fixtures replicate each provider's identifiers (ARN, Azure resource ID, GCP full resource name) because entity resolution depends on them.

CMDB and scanner findings

- CMDB entries are configuration items with relationships; the ServiceNow CMDB lists identification and reconciliation as a core feature, which is what this course rebuilds in small form [7].
- CMDB fixture fields: ci_id, name, class, fqdn, ip, environment, owner_group, criticality, updated_at.
- Scanner findings fixture: finding_id, target (hostname or IP), port, service, title, severity, observed_at, scanner_id.
- Each adapter emits source-native records; normalization happens in the next module.

Instructor demo.

1. Show the adapter interface and one implementation end to end.
2. Show a CT get-entries request shape from RFC 6962 and the equivalent fixture file [3].
3. Show a Resource Graph KQL query and explain why results can be partial [2].
4. Inject a timeout into the DNS fixture server and show bounded retries and a recorded error.
5. Run all five adapters and show counts and errors per source.

Hands-on lab. Build and test the five adapters against fixtures.

1. Implement sources/base.py with the adapter contract and AdapterResult model.
2. Implement the DNS adapter reading fixtures/dns/zone-*.json.
3. Implement the certificate adapter reading fixtures/ct/entries-*.json (log id, index, SANs, issuer, notBefore, notAfter).
4. Implement the cloud adapter reading AWS, Azure and GCP fixture exports.
5. Implement the CMDB and scanner adapters.
6. Add timeouts and bounded retries; test with the fault-injection fixture server.
7. Add a test that each adapter refuses to run without a scope gate.
8. Run all adapters for snapshot 2026-10-27 and store raw results.

Lab success criteria.

- All five adapters return typed records with source, fetched_at and partial flag.
- Fault injection produces recorded errors within the retry limit.
- No adapter can run without the gate.
- Raw record counts match fixtures/expected/raw-counts.json.

Deliverable. Source adapter layer.

Common pitfalls.

- Treating eventually consistent inventory as a point-in-time truth.
- Not flagging partial results from permission gaps.
- Normalizing inside adapters, which hides source-native identifiers.
- Unbounded retries that hammer a source.

GenNoor Tech
For more info - https://gennoor.com

Knowledge check.

1. Why might an Azure Resource Graph query miss resources without an error? Answer: It returns only resources the caller has read access to, with no indication the result is partial [2].
2. How long does Google Cloud Asset Inventory keep change history? Answer: Up to 35 days [10].
3. Which CT endpoints read the latest tree head and the entries? Answer: /ct/v1/get-sth and /ct/v1/get-entries [3].

Slides and assets to prepare.

- Adapter contract diagram.
- Source comparison table (identifiers, history, consistency, permissions).
- Five fixture sets with expected raw counts.
- Fault-injection fixture server.

Sources for this module. 1, 2, 3, 4, 5, 7, 10, 14, 20

Module 3: Asset normalization and canonical IDs

Goal. Normalize all source records into one inventory with canonical asset IDs, first and last seen, confidence, environment, relationships and provenance on every record.

Outline topics.

- Canonical asset IDs; source, first and last seen, confidence, environment and relationships
- Provenance on every record

Learning objectives.

- Learners can design a canonical asset model and ID scheme that is stable across snapshots.
- Learners can normalize hostnames, IPs, certificates and cloud identifiers consistently.
- Learners can maintain first_seen and last_seen per asset and per source.
- Learners can attach provenance and confidence to every attribute.

Asset types and canonical IDs

- Types: domain, hostname, ip, certificate, cloud_resource, service (host:port), ci.
- Canonical ID derives from the strongest native identifier: cloud resource ID, certificate SHA-256 fingerprint, normalized FQDN, IP; prefix with type (host:api-eu2.lab.example).
- Do not derive IDs from mutable attributes (owner, environment).
- The ID algorithm is versioned; changes require a migration.

Normalization rules

- Hostnames: lowercase, strip trailing dot, IDNA encode; DNS name case carries no significance in certificates [15].

GenNoor Tech
TRAIN · INNOVATE · BUILD
for more info - https://gennoor.com

- Wildcard SANs are kept as wildcard records, not expanded, since RFC 5280 leaves wildcard semantics to applications [15].
- IPs: canonical string form from the ipaddress module; classify global vs private [16].
- Certificates: store notBefore/notAfter (inclusive), issuer and SAN list [15].
- Cloud IDs: keep provider-native form; Azure IDs are case-insensitive in practice, so normalize case for matching but store the original.

Provenance, seen times and confidence

- Each attribute has value, source, source_record_id, observed_at; the asset has first_seen and last_seen overall and per source.
- Confidence: observed (directly in a source), derived (computed, e.g. IP from A record), asserted (CMDB claim not observed).
- Environment comes from tags or CMDB with source; conflicts are kept, not resolved silently.
- Relationships (resolves_to, covered_by_cert, hosted_on, owned_by) are edges with their own provenance.

Storing the inventory

- Tables: assets, asset_attributes, relationships, snapshots, raw_records; all keyed by snapshot_id.
- Upsert assets by canonical ID; update last_seen, never overwrite first_seen.
- Keep raw records immutable for audit and reprocessing.
- Inventories built this way are also what BOD 26-04 expects in asset tagging by organization, environment, exposure and type [22].

Instructor demo.

1. Normalize three variants of the same hostname and show one canonical ID.
2. Show a certificate with a wildcard SAN stored as a wildcard record [15].
3. Show provenance for an asset with attributes from DNS, cloud and CMDB.
4. Re-run normalization and show first_seen unchanged and last_seen updated.

Hands-on lab. Normalize all sources into the inventory store.

1. Define the canonical asset, attribute and relationship models.
2. Implement the canonical ID function with tests for each type.
3. Implement hostname, IP, certificate and cloud ID normalizers.
4. Write normalized records with per-attribute provenance.
5. Create relationships from DNS (resolves_to), certs (covered_by_cert) and cloud (hosted_on).
6. Implement first_seen/last_seen upsert logic.
7. Run on snapshot 2026-10-27 and compare with fixtures/expected/normalized.json.
8. Check that 100% of attributes carry provenance.

Lab success criteria.

- Normalized asset count matches the expected file.
- Every attribute and relationship has source and observed_at.
- Re-runs do not change first_seen or create duplicates.
- Canonical ID tests pass for all types.

Deliverable. Normalized asset inventory.

Common pitfalls.

- Expanding wildcard certificates into invented hostnames.
- Using display names as IDs.
- Overwriting first_seen on each run.
- Dropping conflicting attribute values.

Knowledge check.

1. Does RFC 5280 define wildcard matching for dNSName? Answer: No, it leaves wildcard semantics to applications [15].
2. Why version the canonical ID algorithm? Answer: Changing it changes IDs, which breaks history and links unless migrated.
3. What is the difference between observed and asserted confidence? Answer: Observed is seen directly in a technical source; asserted is a claim (e.g. CMDB) not confirmed by observation.

Slides and assets to prepare.

- Canonical asset model diagram.
- Normalization rules card.
- Provenance example record.
- Expected normalized inventory.

Sources for this module. 15, 16, 22

Day 2

Module 4: Entity resolution

Goal. Link records that refer to the same asset using deterministic signals, send uncertain matches to a review queue, and measure link precision.

Outline topics.

- Deterministic signals: exact hostname, certificate SAN, cloud resource ID, verified ownership
- Uncertain matches go to a review queue, never silently merged
- Precision-oriented tests

Learning objectives.

- Learners can rank linking signals by strength and implement deterministic rules.

Genoor Tech TRAIN. INNOVATE. BUILD. For more info - https://genoor.com

- Learners can route uncertain matches to a review queue with evidence instead of merging.
- Learners can write precision-oriented tests against a labeled link set.
- Learners can explain why false merges are worse than missed links in this domain.

Deterministic signals

- Exact normalized hostname across DNS, scanner and CMDB.
- Certificate SAN containing the hostname; SANs are CA-verified identities bound to the key [15].
- Cloud resource ID or ARN present in two sources.
- Verified ownership such as a domain-verification TXT record; Azure App Service asuid TXT records are an example of proof of control [9].
- A CNAME chain links a hostname to a cloud endpoint; follow it within the snapshot only [5].

Weak signals and the review queue

- Weak: similar names (vm-7781 vs db-old), shared IP on shared hosting, same CMDB name with different FQDN.
- Weak signals produce a candidate link with score and evidence, status queued; never auto-merge.
- Review actions: confirm, reject, split; each is audited with the reviewer.
- Shared infrastructure (CDN IPs, load balancers) must not merge different services.

Merge semantics

- A link is an edge same_as with evidence; the inventory view groups linked records, the raw records stay separate.
- Unlinking is always possible because nothing is destroyed.
- Conflicting attributes in a linked group are shown with sources.
- Registrable domain boundaries from the Public Suffix List prevent linking across different registrants under a shared suffix [17].

Precision-oriented testing

- Labeled link set: pairs marked same, different, or ambiguous.
- Link precision = correct auto-links / all auto-links; also report recall and queue size.
- Set the acceptance bar on precision first (e.g. 0.97), then improve recall.
- Add regression cases for every false merge found in review.

Instructor demo.

1. Link a DNS name, a cert SAN and a cloud load balancer to one asset and show the evidence.
2. Show vm-7781 and db-old queued with score 0.61 and evidence.
3. Confirm and reject queue items and show the audit.
4. Run the precision test and show one false merge caused by a shared CDN IP, then fix the rule.

Hands-on lab. Implement entity resolution with a review queue and measure link precision.

1. Implement deterministic rules (hostname, SAN, cloud ID, verified ownership, CNAME chain).
2. Implement candidate generation for weak signals with scores and evidence.
3. Create the review queue table and API endpoints for confirm, reject and split.
4. Exclude shared-infrastructure IPs from IP-based linking using the fixture list.
5. Run on the snapshot and compute precision and recall against fixtures/evals/links.json.
6. Review five queued items and record decisions.
7. Add a regression test for any false merge found.

Lab success criteria.

- Link precision meets the course bar on the labeled set.
- No weak-signal match is merged without review.
- Every link has evidence and can be undone.
- Review actions are audited.

Deliverable. Entity resolution with review queue.

Common pitfalls.

- Merging on shared IPs.
- Fuzzy matching auto-merges.
- Destroying raw records on merge.
- Measuring only recall.

Knowledge check.

1. Why is a certificate SAN a strong linking signal? Answer: The CA must verify SAN identities, binding them to the certificate key [15].
2. What happens to a 0.61 confidence match? Answer: It goes to the review queue with its evidence; it is never merged automatically.
3. Why prefer precision? Answer: A false merge assigns wrong owners and exposure to an asset, which misleads every later step.

Slides and assets to prepare.

- Signal strength ladder.
- Labeled link evaluation set.
- Review queue UI mock.
- False merge case study (shared CDN IP).

Sources for this module. 5, 9, 15, 17

Module 5: Change detection

Goal. Compare inventory snapshots to classify assets as new, changed, missing, stale or potentially orphaned, validated against fixtures with known outcomes.

Gennoor Tech
INNOVATE. BUILD.
For more info - https://gennoor.com

Outline topics.

- Snapshot comparison: new, changed, missing, stale and potentially orphaned assets
- Synthetic fixtures with known expected outcomes

Learning objectives.

- Learners can implement snapshot diffing on canonical IDs and attributes.
- Learners can define stale and orphaned with explicit rules and windows.
- Learners can relate changes to real risks such as dangling DNS and expiring certificates.
- Learners can validate change detection against expected outcomes.

Snapshot comparison

- New: canonical ID present now, absent before. Missing: present before, absent now in a source that covered it.
- Changed: monitored attribute differs (cert issuer, IP, CNAME target, open ports).
- Coverage matters: a source that failed or returned partial results cannot make anything 'missing' [2].
- Cloud sources have their own change feeds: Azure resourcechanges with changeType Create, Update, Delete and previous/new values [19]; AWS Config history [1]; GCP 35-day history [10].

Stale and orphaned

- Stale: not seen by any technical source for longer than a window (e.g. 30 days) but still in CMDB or DNS.
- Potentially orphaned: has DNS or certificate presence but no owner and no matching cloud resource.
- Dangling DNS is the critical case: a CNAME pointing to a deprovisioned resource can be taken over, and the attacker can get a valid certificate [9].
- Label all of these 'potential'; they become review tickets, not automatic deletions.

Certificate changes

- Track issuer change, SAN change, and days to notAfter [15].
- Maximum public TLS certificate validity is falling from 398 to 47 days between March 2026 and March 2029, so expiry and reissue events will be frequent [21].
- New certificates for your domains appear in CT logs; a monitor inspects every new entry [4].
- An unexpected issuer change is a 'changed' event worth review, not proof of compromise.

Validation with known outcomes

- Fixtures: snapshot pairs with an answer key per asset (new, changed, missing, stale, orphaned, unchanged).
- Change-detection accuracy = correct classifications / total assets in the key.
- Include traps: source outage, renamed host with same cloud ID, cert renewal with same SANs.

- BOD 23-01's seven-day discovery cadence is a useful reference for choosing snapshot frequency [13].

Instructor demo.

1. Diff snapshots 10-27 and 10-28 and show the five classes.
2. Show legacy-sso.lab.example missing 41 days and flagged as potentially stale.
3. Show a dangling CNAME fixture and explain the takeover path [9].
4. Simulate a DNS source outage and show nothing marked missing.
5. Query the Azure resourcechanges fixture for deletes and correlate with DNS [19].

Hands-on lab. Build snapshot comparison and validate it against the fixture set.

1. Implement the snapshot diff on canonical IDs and monitored attributes.
2. Implement coverage checks per source per snapshot.
3. Implement stale and orphaned rules with configurable windows.
4. Implement the dangling DNS check: CNAME target with no matching cloud resource.
5. Implement certificate expiry and issuer-change detection.
6. Run against fixtures/snapshots/ pairs and compute accuracy against the answer key.
7. Investigate every misclassification and fix or document.
8. Write a change report for the 10-27 to 10-28 pair.

Lab success criteria.

- Accuracy meets the course bar on the answer key.
- Source outages never produce missing classifications.
- Dangling CNAME fixtures are detected.
- Every classification lists the evidence that triggered it.

Deliverable. Change detection engine.

Common pitfalls.

- Marking assets missing when the source failed.
- Deleting stale assets automatically.
- Treating every certificate renewal as a change.
- Ignoring cloud deletes when DNS still points to the resource.

Knowledge check.

1. Why is a CNAME to a deleted cloud resource dangerous? Answer: An attacker can provision the same FQDN and receive the subdomain's traffic, even with a valid certificate [9].
2. Which Azure field should you use to classify a change? Answer: changeType (Create, Update, Delete), not operation [19].
3. What will the maximum public TLS certificate validity be by March 2029? Answer: 47 days [21].

Slides and assets to prepare.

GenMoor Tech INNOVATE BUILD. For more info - https://genmoor.com

- Change class definitions card.
- Snapshot fixture pairs with answer keys.
- Dangling DNS takeover diagram.
- Change report template.

Sources for this module. 1, 2, 4, 9, 10, 13, 15, 19, 21

Module 6: Exposure correlation with an agent

Goal. Join assets to scanner findings, service metadata and business criticality, and have an agent summarize exposure while strictly separating observed facts from inferred exposure, with unsupported claims measured.

Outline topics.

- Joining assets to scanner findings, service metadata and business criticality
- The agent summarizes exposure but must distinguish observed facts from inferred exposure
- Unsupported exposure claims are measured

Learning objectives.

- Learners can correlate findings and services to canonical assets.
- Learners can define observed versus inferred exposure with examples.
- Learners can build an agent with a strict schema whose every claim cites evidence IDs.
- Learners can measure the unsupported-claim rate and reduce it.

Correlation

- Join scanner findings by target hostname or IP to canonical assets through resolved links; unmatched findings are reported.
- Attach service metadata (port, protocol, TLS version) as observed attributes with scanner ID and time.
- Attach criticality from CMDB as asserted, with source [7].
- Internet-facing means observed from an external vantage point in the fixture, not inferred from a name; BOD 26-04 treats an asset as exposed if any method finds it exposed [22].

Observed versus inferred

- Observed: port 443 open (scanner#7), TLS 1.2 negotiated, two medium findings.
- Inferred: 'admin path exposed' because a cert SAN contains admin.; plausible but not observed.
- Unknown: compensating controls, authentication on the endpoint.
- Information gathering techniques such as entry-point identification and architecture mapping (WSTG 4.1) produce observations only when actually performed in scope [8].

The exposure agent

- Input: asset record, linked evidence with IDs, findings and criticality; no free browsing and no discovery tools.
- Output schema: summary, observed[] (claim, evidence_ids), inferred[] (claim, basis, evidence_ids), unknown[], review_needed.
- Use JSON schema outputs; set additionalProperties false and check stop_reason before parsing [24].
- Validator: each observed claim must cite an evidence ID of an observing source; claims citing only a cert SAN or a name must be in inferred.
- OWASP recommends grounding, automatic validation and labeling AI-generated content [23].

Measuring unsupported claims

- Unsupported claim: an observed claim without valid evidence, or any claim not traceable to input.
- Rate = unsupported claims / total claims over the evaluation set, run several times.
- Flag, do not drop: flagged claims are shown with a marker in the report.
- Track the rate per prompt version.

Instructor demo.

1. Correlate findings for api-eu2.lab.example and show observed attributes with sources.
2. Run the agent and show observed, inferred and unknown sections.
3. Show the 'admin path exposed' claim flagged as inferred from SAN only.
4. Plant a misleading string in a finding title and show the validator catching an unsupported observed claim.
5. Show the unsupported-claim rate across five runs.

Hands-on lab. Correlate exposure and review the agent's output for unsupported claims.

1. Implement correlation of findings and services to canonical assets.
2. Report unmatched findings.
3. Define the exposure output schema as Pydantic and JSON schema.
4. Write the agent prompt with the observed/inferred/unknown rules.
5. Implement the claim validator using evidence source types.
6. Run on the evaluation set five times and compute the unsupported-claim rate.
7. Review ten outputs by hand and compare with the validator.
8. Improve the prompt or validator and re-run.

Lab success criteria.

- Every observed claim cites an observing source.
- The unsupported-claim rate is measured and reported per prompt version.
- Unmatched findings are listed, not dropped.
- Manual review agrees with the validator on the sampled outputs, with disagreements documented.

Gernoor Tech
For more info - https://gernoor.com

Deliverable. Exposure correlation.

Common pitfalls.

- Treating 'internet-facing' as true because a hostname looks public.
- Letting the agent call discovery tools.
- Dropping flagged claims silently.
- Evaluating one run.

Knowledge check.

1. Is 'admin path exposed' observed if the only evidence is a SAN? Answer: No, it is inferred and must be labeled so.
2. What must you check before parsing a structured response? Answer: stop_reason, because refusal or max_tokens output may not match the schema [24].
3. Define the unsupported-claim rate. Answer: Claims not supported by supplied evidence divided by all claims.

Slides and assets to prepare.

- Observed vs inferred vs unknown examples sheet.
- Exposure output schema.
- Evaluation set with labeled claims.
- Unsupported-claim dashboard panel.

Sources for this module. 7, 8, 22, 23, 24

Day 3

Module 7: Ownership resolution

Goal. Suggest a responsible team for each asset from synthetic CMDB records, tags and repository metadata, with evidence attached and manual correction supported.

Outline topics.

- Synthetic CMDB records, tags and repository metadata to suggest a responsible team
- Evidence attached to every suggestion; manual correction supported

Learning objectives.

- Learners can combine ownership signals with explicit precedence and confidence.
- Learners can attach evidence to every suggestion.
- Learners can implement manual correction that persists and feeds back into evaluation.
- Learners can measure the unowned asset rate.

Ownership signals

Gennoor Tech TRAIN · INNOVATE · BUILD
For more info - https://gennoor.com

- CMDB owner group on the CI [7].
- Cloud tags (e.g. team=platform-eu); AWS tag keys and values are case sensitive, and AWS recommends a standard format enforced with tag policies [18].
- Repository CODEOWNERS for the infrastructure code that defines the resource; owners are @user or @org/team and the last matching pattern wins [12].
- DNS zone or certificate requester as weak signals.

Precedence and confidence

- Default precedence: verified manual correction > CMDB owner (if recent) > enforced tag > CODEOWNERS > weak signals.
- Agreement of two independent signals raises confidence; disagreement creates a conflict for review.
- Normalize tag keys for matching (Team, team) but report the inconsistency [18].
- No signal: owner unknown, routed to triage. Never guess.

Evidence and correction

- Each suggestion lists signals, values, sources and timestamps.
- Manual corrections record who, why and when, and override until the evidence changes.
- Corrections become labeled data for the ownership evaluation set.
- Tags must not contain personal or sensitive data; record team names, not individuals [18].

Ownership metrics

- Unowned asset rate = assets with owner unknown / all in-scope assets.
- Suggestion accuracy against the labeled set.
- Conflict rate between signals per source pair.
- BOD 26-04's tagging requirement (organization, environment, exposure, asset type) shows ownership is part of exposure management [22].

Instructor demo.

1. Show api-eu2.lab.example with tag team=platform-eu and CODEOWNERS @lab-org/platform-eu agreeing.
2. Show a conflict between CMDB and tag and the review item.
3. Correct a wrong suggestion and show it persists across a rerun.
4. Show unowned rate before and after corrections.

Hands-on lab. Resolve ownership for the inventory and correct three wrong suggestions.

1. Implement signal extractors for CMDB, tags and CODEOWNERS fixtures.
2. Implement a CODEOWNERS pattern matcher with last-match-wins semantics.
3. Implement precedence and confidence rules.
4. Store suggestions with evidence.
5. Implement the correction API with audit.

GenNoor Tech
For more info: https://gennoor.com

6. Run, then find and correct three wrong suggestions using the answer key.
7. Compute unowned rate and suggestion accuracy.

Lab success criteria.

- Every suggestion has evidence; no owner is assigned without a signal.
- Three corrections persist across reruns and are audited.
- Unowned rate and accuracy are reported.
- Conflicts are routed to review.

Deliverable. Ownership resolver.

Common pitfalls.

- First-match CODEOWNERS logic.
- Treating tag case variants as different owners.
- Assigning a default owner instead of unknown.
- Losing corrections on rerun.

Knowledge check.

1. In CODEOWNERS, which rule wins when several match? Answer: The last matching pattern [12].
2. Are AWS tag keys case sensitive? Answer: Yes; keys and values are case sensitive [18].
3. What should happen when no ownership signal exists? Answer: Owner unknown, routed to triage, counted in the unowned rate.

Slides and assets to prepare.

- Ownership signal precedence chart.
- CMDB, tag and CODEOWNERS fixtures with answer key.
- Correction workflow diagram.

Sources for this module. 7, 12, 18, 22

Module 8: Prioritization, ticketing and inventory quality metrics

Goal. Rank exposures with transparent criteria, raise review tickets through a mock connector, and publish inventory quality metrics with an asset graph and audit report.

Outline topics.

- Transparent ranking criteria and review tickets through a mock connector
- Entity-link precision, unowned asset rate, stale-asset classification, change-detection accuracy
- Asset graph, reports and audit

Learning objectives.

- Learners can implement transparent, rule-based ranking with visible inputs.
- Learners can raise idempotent review tickets with owner and evidence.

GenNoor Tech
TRAIN. INNOVATE. BUILD.
For more info - https://gennoor.com

- Learners can compute link precision, unowned rate, stale classification accuracy and change-detection accuracy.
- Learners can produce an asset graph and an audit report.

Transparent ranking

- Inputs: observed exposure, criticality, finding severity, change class (new, dangling), ownership state, evidence quality.
- Rules as data with points and rationale; inferred exposure counts less than observed.
- Dangling DNS and newly exposed critical assets rank high because of takeover risk [9].
- Public exposure is a primary factor in federal prioritization under BOD 26-04 [22].

Review tickets through a mock connector

- Ticket types: review exposure, confirm decommission, assign owner, review match.
- Jira-shaped mock: POST /rest/api/2/issue with project.key, summary, description and issuetype.name; returns id, key, self [25].
- Idempotency: ticket_key from (asset canonical ID, ticket type); search by label with JQL before create [25].
- Tickets are for review; the platform never changes DNS or cloud resources.

Inventory quality metrics

- Entity-link precision (from the resolution module).
- Unowned asset rate; stale classification accuracy; change-detection accuracy.
- Scope events: blocked and throttled counts per run, and audit entry count.
- Trend over snapshots, not just a single value.

Asset graph, reports and audit

- Graph nodes are canonical assets; edges are relationships and same_as links with provenance.
- Export to a simple JSON graph and render in the UI; filter by owner and exposure.
- Report: top exposures, changes, unowned, scope events, metrics, limitations.
- Audit report lists every scope decision and manual correction for the run.

Instructor demo.

1. Show the ranking rules and the computed rank for ASM-301 and ASM-302.
2. Raise tickets in the mock and re-run to show no duplicates.
3. Show the metrics panel and the graph view.
4. Show the audit report including the blocked partner-corp.example target.

Hands-on lab. Rank, raise review tickets and publish the quality metrics.

1. Implement ranking rules as data.
2. Generate review items per type.
3. Implement the Jira-shaped mock connector client with label-based idempotency.

4. Raise tickets and re-run to confirm zero duplicates.
5. Compute the four quality metrics plus scope event counts.
6. Export the asset graph JSON and render it.
7. Write the run report and audit report.

Lab success criteria.

- Rankings match the expected order and each shows its inputs.
- Re-runs create no duplicate tickets.
- All four quality metrics are computed against fixtures.
- Reports include limitations and scope events.

Deliverable. Metrics report and asset graph.

Common pitfalls.

- Ranking inferred exposure the same as observed.
- Tickets without owner or evidence.
- Metrics computed on a different snapshot than the report.
- Letting tickets trigger automatic changes.

Knowledge check.

1. Which call creates an issue in the Jira-shaped API? Answer: POST /rest/api/2/issue with fields project.key, summary, description and issuetype.name [25].
2. Name the four inventory quality metrics. Answer: Entity-link precision, unowned asset rate, stale classification accuracy, change-detection accuracy.
3. Why rank dangling DNS high? Answer: Because it allows subdomain takeover [9].

Slides and assets to prepare.

- Ranking rules table.
- Ticket templates per type.
- Metrics dashboard layout.
- Report and audit templates.

Sources for this module. 9, 22, 25

Module 9: Capstone: scoped attack surface platform

Goal. Run approved sources to owned, prioritized exposures in one repeatable run, with every discovery action constrained to scope, and present acceptance evidence and limitations.

Outline topics.

- Approved sources to owned, prioritized exposures in one run
- Acceptance criteria, limitations, demo

Gennoor Tech TRAINING INNOVATE BUILD For more info - https://gennoor.com

Learning objectives.

- Learners can run the full platform from a clean environment with one command.
- Learners can show evidence for every acceptance criterion.
- Learners can explain limitations and the boundary between the lab and real deployments.
- Learners can demonstrate a blocked scope violation during the demo.

Acceptance criteria

- All discovery is constrained to approved scope; violations are blocked and logged.
- Every asset record includes provenance and timestamps.
- Uncertain matches are queued, not merged; new/changed/stale logic is tested against fixtures.
- Ownership suggestions include evidence and can be corrected; metrics report gaps and correlation quality.

Limitations and deployment boundary

- Real cloud inventories are eventually consistent and permission-limited [2][10].
- Live CT monitoring must process every new entry of every watched log, which is an operational commitment [4].
- Active checks outside fixtures need a signed ROE with exclude list and halt criteria [6].
- The agent is advisory; scope, ranking and ownership precedence stay deterministic.

Demo structure

- Show the scope policy and one blocked target.
- Run the pipeline, then trace one asset from raw records to ticket.
- Show a change report with a dangling DNS case.
- Close with metrics and limitations.

Instructor demo.

1. Run make reset && make run-all on a clean clone.
2. Trace one asset through all tables and the graph.
3. Show the audit report with the blocked target.
4. Model the five-minute demo.

Hands-on lab. Capstone build and demo of the end-to-end attack surface management lab.

1. Reset and run the full pipeline with one command on the capstone snapshot pair.
2. Fix failures; the final run must need no manual edits.
3. Collect acceptance evidence in docs/acceptance.md.
4. Update README, runbook, architecture diagram, data-flow notes and limitations.
5. Include one deliberate out-of-scope attempt in the demo script.
6. Prepare and deliver the five-minute demo.

Lab success criteria.

Gemnoor Tech INNOVATE. BUILD. TRUST. For more info - https://gemnoor.com

- The run completes from a clean environment without manual edits.
- Every acceptance criterion has linked evidence.
- The demo shows a blocked and logged out-of-scope target.
- Limitations are specific.

Deliverable. End-to-end attack surface management lab.

Common pitfalls.

- Demo depends on manual steps.
- Scope violation demo uses a real third-party domain.
- Metrics without the snapshot they refer to.

Knowledge check.

1. What must be in place before any active check outside fixtures? Answer: Signed rules of engagement with scope, exclude list and halt criteria [6].
2. Why can a cloud inventory snapshot be incomplete? Answer: Eventual consistency and permission-limited results [2][10].
3. Which decisions stay deterministic? Answer: Scope, ranking and ownership precedence.

Slides and assets to prepare.

- Capstone snapshot pair with expected outcomes.
- Acceptance evidence template.
- Rubric (functionality 25%, safety controls 20%, testing and evaluation 20%, engineering 15%, explainability 10%, documentation 10%).
- Demo checklist.

Sources for this module. 2, 4, 6, 10

6. Capstone plan

A platform that consolidates approved asset sources, detects changes and inventory gaps, links assets to owners and findings, and prioritizes exposure review, with every discovery action constrained to authorized scope.

Build a platform that consolidates approved asset sources, detects changes and inventory gaps, links assets to owners and findings, and prioritizes exposure review, with every discovery action constrained to authorized scope, demonstrated on a new capstone snapshot pair.

Milestones

1. M1: scope policy loaded and gate tests green; out-of-scope attempt blocked and logged.
2. M2: five adapters and normalized inventory with 100% provenance.
3. M3: entity resolution with review queue meeting the precision bar; change report for the capstone pair.

Gennoor Tech TRAINING INNOVATE BUILD for more info - https://gennoor.com

4. M4: exposure summaries with measured unsupported-claim rate; ownership with corrections.
5. M5: ranked review tickets, metrics, asset graph, audit report and five-minute demo.

Evaluation notes

- Use the client rubric: working functionality 25%, security and safety controls 20%, testing and evaluation 20%, engineering quality 15%, evidence and explainability 10%, documentation and presentation 10%.
- Automatic fail on any adapter action that bypasses the scope gate or any silent merge.
- Metrics must be computed on the capstone fixtures, not the training snapshots.
- Reward clear limitations and honest unsupported-claim rates.

Assessment weights

Assessment area	Weight	Evidence
Working functionality	25%	Demonstrable end-to-end workflow in a repeatable lab
Security and safety controls	20%	Threat model, authorization boundaries, approval gates, safe failure
Testing and evaluation	20%	Automated tests, golden dataset, measured quality and known limitations
Engineering quality	15%	Clear interfaces, error handling, reproducible setup, maintainable code
Evidence and explainability	10%	Traceable findings, sources, assumptions, logs
Documentation and presentation	10%	Architecture diagram, README, runbook, demo, lessons learned

7. Promotional video: research behind the script

Opening scenario. 'You cannot protect what you cannot find' reflects the asset-visibility premise of BOD 23-01 [13]. Forgotten subdomains are realistic because dangling DNS records enable subdomain takeover [9]; expired certificates are realistic because certificate lifetimes are shrinking and every issued certificate is public in CT logs [21][20]; 'cloud resources nobody owns' matches the ownership tagging that BOD 26-04 now requires of agencies [22].

Claim or on-screen text	Basis
5 sources, one graph: DNS, certificates, cloud inventory, CMDB and scanner findings	Course design from the client's five adapters; each source type is real and documented [1][3][5][7].
Weak scope is the biggest risk in attack surface tooling	A course position from the client's safety note, not a measured statistic; consistent with NIST's requirement for ROE with an exclude list before testing [6].
BLOCKED: target 'partner-corp.example' not in scope (logged, audit#2231)	Synthetic lab output; uses a reserved example. name [11]. Behavior taught in the scope module.
dns: 412 records, certs: 96, cloud: 1,204, cmdb: 388, findings: 2,019; normalized: 1,733 assets	Illustrative fixture counts, not real data.
NEW api-eu2.lab.example (dns + cert SAN + cloud lb-eu2)	Teaching scenario showing deterministic linking signals; SANs are CA-verified identities [15].
CHANGED shop.lab.example cert issuer changed	Teaching scenario; issuer changes are visible in CT and the certificate issuer field [4][15].
MISSING legacy-sso.lab.example last seen 41d ago → STALE?	Teaching scenario; the 30-day stale window is a course default, not a standard.
REVIEW vm-7781 ↔ 'db-old' match confidence 0.61 (queued)	Synthetic; demonstrates the client's rule that uncertain matches are queued, not merged.
observed: TLS 1.2, port 443 open … inferred: admin path exposed (based on cert SAN only) ← flagged	Teaching scenario for observed vs inferred exposure; validation approach supported by OWASP guidance [23].
owner: platform-eu (tag team=platform-eu, repo CODEOWNERS) confidence 0.92	Synthetic; ownership signals from cloud tags and CODEOWNERS are real mechanisms [18][12].
metrics: link precision 0.97, unowned 3.1%, stale accuracy 0.95, change accuracy 0.98	Illustrative lab results, not benchmarks.
0 out-of-scope discovery	A property of the lab design (fixture-only sources, blocked outbound network, scope gate), demonstrated by tests rather than a general claim.

References

[1] What Is AWS Config?. Amazon Web Services.
https://docs.aws.amazon.com/config/latest/developerguide/WhatIsConfig.html
Supports: Source adapters; Change detection. AWS Config records resource configurations and relationships and how they changed over time, can notify on create/modify/delete, offers advanced queries over current configuration state, and an aggregator collects configuration and compliance data from multiple accounts and Regions into one account and Region.

[2] Overview of Azure Resource Graph. Microsoft Learn. https://learn.microsoft.com/en-us/azure/governance/resource-graph/overview
Supports: Source adapters; Asset normalization and canonical IDs. Azure Resource Graph queries resource properties across subscriptions with a KQL-based language, returns results only for resources the caller has at least read RBAC access to (partial results give no warning), keeps 14 days of change history, is not strongly consistent, and throttles per user with x-ms-user-quota-remaining and x-ms-user-quota-resets-after headers.

[3] RFC 6962: Certificate Transparency. IETF (RFC Editor). https://www.rfc-editor.org/rfc/rfc6962
Supports: Source adapters. CT logs are publicly auditable, append-only, untrusted Merkle-tree logs of issued certificates; precertificates carry a poison extension; clients read entries with GET /ct/v1/get-entries?start=&end= (0-based) and the latest tree head with /ct/v1/get-sth; the RFC is Experimental and obsoleted by RFC 9162.

[4] RFC 9162: Certificate Transparency Version 2.0. IETF (RFC Editor). https://www.rfc-editor.org/rfc/rfc9162.html
Supports: Source adapters; Change detection. Monitors watch logs for correct behavior or certificates of interest, such as all certificates for a domain, and "MUST at least inspect every new entry in every log it watches"; RFC 9162 is Experimental and obsoletes RFC 6962.

[5] RFC 1034: Domain Names - Concepts and Facilities. IETF (RFC Editor). https://www.rfc-editor.org/rfc/rfc1034
Supports: Source adapters; Entity resolution. A zone is authoritative for a connected region of the name space; the order of RRs in a set has no significance; TTL limits how long an RR may be cached; and if a CNAME RR is present at a node no other data should be present.

[6] NIST SP 800-115: Technical Guide to Information Security Testing and Assessment. NIST. https://nvlpubs.nist.gov/nistpubs/Legacy/SP/nistspecialpublication800-115.pdf
Supports: Authorization, scope policy and the lab. Rules of Engagement are detailed guidelines and constraints established before testing that give the team authority to perform defined activities without further permission; the Appendix B template covers scope, test schedule and hours, personnel and points of contact, incident handling with halt criteria, and target systems including authorized and unauthorized addresses and an exclude list.

[7] Configuration Management Database (CMDB). ServiceNow. https://www.servicenow.com/docs/r/zurich/servicenow-platform/configuration-management-database-cmdb/c_ITILConfigurationManagement.html
Supports: Source adapters; Ownership resolution. The ServiceNow CMDB builds logical representations of assets, services and the relationships between them, stores component details as configuration items, and lists CMDB Identification and Reconciliation as a core feature; CMDB 360 tracks how multiple sources populate the same CI attributes.

[8] WSTG v4.2: 4.1 Information Gathering. OWASP. https://wstg.owasp.org/v4.2/4-Web_Application_Security_Testing/01-Information_Gathering/
Supports: Authorization, scope policy and the lab; Exposure correlation with an agent. WSTG section 4.1 lists ten information gathering tests, from search engine discovery and web server fingerprinting to enumerating applications on a web server, identifying entry points and mapping application architecture.

[9] Prevent dangling DNS entries and avoid subdomain takeover. Microsoft Learn. https://learn.microsoft.com/en-us/azure/security/fundamentals/subdomain-takeover
Supports: Change detection; Exposure correlation; promo claims. A DNS record (especially a CNAME) pointing to a deprovisioned resource is a dangling entry that lets an attacker provision the same FQDN and take over the subdomain, including obtaining a valid TLS certificate; Microsoft recommends removing such records, using alias records and asuid TXT domain verification, adding 'Remove DNS entry' to decommission checklists, and keeping a service catalog of FQDN endpoints and owners.

[10] Cloud Asset Inventory overview. Google Cloud. https://docs.cloud.google.com/asset-inventory/docs/overview
Supports: Source adapters; Change detection. Cloud Asset Inventory searches, exports (to BigQuery or Cloud Storage) and monitors asset metadata via feeds; create, update and delete history is kept for up to 35 days; it is eventually consistent and not intended for live querying; relationships require Security Command Center Premium or Enterprise.

[11] RFC 6761: Special-Use Domain Names. IETF (RFC Editor). https://www.rfc-editor.org/rfc/rfc6761
Supports: Authorization, scope policy and the lab. example., example.com/net/org are reserved for documentation and resolve normally; test. names get immediate negative answers from caching servers by default; invalid. always returns NXDOMAIN; registrars must not register any of these; RFC 6761 updates RFC 2606.

[12] About code owners. GitHub Docs. https://docs.github.com/en/repositories/managing-your-repositorys-settings-and-features/customizing-your-repository/about-code-owners
Supports: Ownership resolution. CODEOWNERS lives in .github/, the root or docs/ (first found wins), uses gitignore-style patterns where the last matching pattern takes precedence, names owners as @user or @org/team who must have write access, must be under 3 MB, and invalid lines are skipped.

[13] BOD 23-01: Improving Asset Visibility and Vulnerability Detection on Federal Networks. CISA. https://www.cisa.gov/news-events/directives/bod-23-01-improving-asset-visibility-and-vulnerability-detection-federal-networks
Supports: Why this course; Change detection. BOD 23-01 requires automated asset discovery every 7 days covering at least the agency's IPv4 space and vulnerability enumeration every 14 days; asset discovery is defined as non-intrusive identification of network-addressable assets, and scope covers IPv4 and IPv6 assets on-premises, roaming and cloud, excluding ephemeral assets and third-party SaaS.

[14] LLM06:2025 Excessive Agency. OWASP GenAI Security Project. https://genai.owasp.org/llmrisk/llm062025-excessive-agency/
Supports: Authorization, scope policy and the lab; Source adapters. Excessive agency arises from excessive functionality, permissions or autonomy; mitigations include minimal and narrow tools (avoid open-ended URL or shell tools), least-privilege credentials, user approval for high-impact actions, complete mediation of authorization in downstream systems, logging and rate limiting.

[15] RFC 5280: Internet X.509 PKI Certificate and CRL Profile. IETF (RFC Editor). https://www.rfc-editor.org/rfc/rfc5280
Supports: Entity resolution; Asset normalization. The subjectAltName extension binds DNS names, IP addresses and other identities to the certificate and must be fully verified by the CA; dNSName uses RFC 1034 preferred name syntax with no significance to case, wildcard semantics are left to applications, and validity is expressed as inclusive notBefore and notAfter times.

[16] ipaddress: IPv4/IPv6 manipulation library. Python Software Foundation. https://docs.python.org/3/library/ipaddress.html
Supports: Authorization, scope policy and the lab. ip_address(x) in ip_network(y) tests membership; subnet_of, supernet_of and overlaps compare networks; ip_network is strict by default and raises ValueError when host bits are set; is_global and is_private follow IANA special-purpose registries, with 100.64.0.0/10 neither private nor global.

[17] Public Suffix List. Mozilla / Public Suffix List community. https://publicsuffix.org/
Supports: Authorization, scope policy and the lab; Entity resolution. The Public Suffix List lists suffixes such as com or co.uk under which people can (or historically could) register names, and is used by browsers to stop cookies being set on public suffixes and to group sites; it is maintained by registries and the community.

[18] Tagging best practices (Tag Editor user guide). Amazon Web Services. https://docs.aws.amazon.com/tag-editor/latest/userguide/best-practices-and-strats.html
Supports: Ownership resolution. AWS advises not to put PII or sensitive data in tags, to use a standardized case-sensitive format (keys and values are case sensitive), to enforce standards with tag policies in AWS Organizations, and notes a 50 user-tag limit per resource, 128-character keys and 256-character values.

[19] Get resource changes - Azure Resource Graph. Microsoft Learn. https://learn.microsoft.com/en-us/azure/governance/resource-graph/how-to/get-resource-changes
Supports: Change detection. Each create, update or delete produces a Microsoft.Resources/changes record (usually within five minutes) with targetResourceId, changeType, changeAttributes (timestamp, changedBy, clientType, operation, previous/new snapshot IDs) and per-property previousValue/newValue, queryable from the resourcechanges table; Microsoft recommends using changeType rather than operation to classify changes.

[20] Certificate Transparency (CT) Logs. Let's Encrypt. https://letsencrypt.org/docs/ct-logs/
Supports: Source adapters. Let's Encrypt submits all certificates it issues to CT logs and operates its own logs (Sycamore and Willow in production, Twig for testing).

[21] Ballot SC-081v3: Introduce Schedule of Reducing Validity and Data Reuse Periods. CA/Browser Forum. https://cabforum.org/2025/04/11/ballot-sc081v3-introduce-schedule-of-reducing-validity-and-data-reuse-periods/
Supports: Change detection; promo claims. Ballot SC-081v3 reduces maximum TLS subscriber certificate validity from 398 days to 47 days in steps from March 2026 to March 2029, and SAN validation data reuse from 398 to 10 days; it passed 25-0 among certificate issuers and 4-0 among consumers.

[22] BOD 26-04: Prioritizing Security Updates Based on Risk. CISA. https://www.cisa.gov/news-events/directives/bod-26-04-prioritizing-security-updates-based-risk
Supports: Prioritization, ticketing and inventory quality metrics; Ownership resolution. BOD 26-04 makes asset exposure (an asset counts as exposed if any method finds it exposed) one of four remediation-timeline inputs, and requires agencies to continuously identify and tag externally reachable assets by organization, environment, exposure and asset type.

[23] LLM09:2025 Misinformation. OWASP GenAI Security Project. https://genai.owasp.org/llmrisk/llm092025-misinformation/
Supports: Exposure correlation with an agent. OWASP identifies hallucination and overreliance as causes of LLM misinformation and recommends grounding in trusted sources, cross-verification and human oversight, automatic validation of key outputs and labeling AI-generated content.

[24] Structured outputs. Anthropic (Claude API docs). https://platform.claude.com/docs/en/build-with-claude/structured-outputs
Supports: Exposure correlation with an agent. JSON outputs use output_config.format with type json_schema (strict tools use strict: true); objects require additionalProperties false, numeric and string-length constraints are not supported, and refusal or max_tokens stop reasons can yield output that does not match the schema.

[25] JIRA REST API examples. Atlassian Developer. https://developer.atlassian.com/server/jira/platform/jira-rest-api-examples/
Supports: Prioritization, ticketing and inventory quality metrics. Create issues with POST /rest/api/2/issue using fields.project.key, summary, description and issuetype.name (response: id, key, self) and search with GET /rest/api/2/search?jql=... returning startAt, maxResults, total and issues.

For more info - https://gennoor.com

Gennoor Tech
For more info - https://gennoor.com